

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины «Программирование на Python»**

Выполнил:
Оразов Тимур
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Р. А.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

Вариант 16

Тема: Работа со списками и кортежами в языке Python

Цель: приобретение навыков по работе со списками и кортежами при написании программ с помощью языка программирования Python версии 3.x.

Ссылка на репозиторий с которого выполнялась работа:

https://github.com/Tim120903/laba_4

Ход работы:

The screenshot shows the 'Create a new repository' page on GitHub. It is divided into two sections: 'General' and 'Configuration'.

General Section:

- Owner *:** A dropdown menu showing 'Tim120903'.
- Repository name *:** A text input field containing 'laba4'. Below the field, a green checkmark indicates 'laba4 is available.'
- Description:** An empty text area with a character count '0 / 350 characters'.

Configuration Section:

- Choose visibility *:** A dropdown menu set to 'Public'.
- Add README:** A toggle switch is turned 'On'.
- Add .gitignore:** A dropdown menu set to 'Python'.
- Add license:** A dropdown menu set to 'MIT License'.

A green 'Create repository' button is located at the bottom right of the form.

Рис. 1- создание общедоступного репозитория.

```
C:\Users\Тимур>git clone https://github.com/Tim120903/laba_4.git
```

Рис. 2 - копирование репозитория на устройство.

```
208
209 # Covers JetBrains IDEs: IntelliJ, GoLand, RubyMine, PhpStorm, AppCode, PyCharm
210 # Reference: https://intellij-support.jetbrains.com/hc/en-us/articles/206544839
211
212 # User-specific stuff
213 .idea/**/workspace.xml
214 .idea/**/tasks.xml
215 .idea/**/usage.statistics.xml
216 .idea/**/dictionaries
217 .idea/**/shelf
218
219 # AWS User-specific
220 .idea/**/aws.xml
221
222 # Generated files
223 .idea/**/contentModel.xml
224
225 # Sensitive or high-churn files
226 .idea/**/dataSources/
227 .idea/**/dataSources.ids
228 .idea/**/dataSources.local.xml
229 .idea/**/sqlDataSources.xml
230 .idea/**/dynamic.xml
231 .idea/**/uiDesigner.xml
232 .idea/**/dbnavigator.xml
233
234 # Gradle
235 .idea/**/gradle.xml
236 .idea/**/libraries
237
238 # Gradle and Maven with auto-import
239 # When using Gradle or Maven with auto-import, you should exclude module files,
240 # since they will be recreated, and may cause churn. Uncomment if using
241 # auto-import.
```

Рис. 3 - Добавленные правила в файл .gitignore

```
Primer_1.py x
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import sys
5
6
7  if __name__ == '__main__':
8      A = tuple(map(int, input().split()))
9      if len(A) != 10:
10         print('Неверный размер кортежа', file=sys.stderr)
11         exit(1)
12
13         s = 0
14         for item in A:
15             if abs(item) < 5:
16                 s += item
17
18         print(s)
19
20 Run Primer_1 x
21
22
23 C:\Users\Тимур\laba_4\.venv\Scripts\python.exe C:\Users\
24 1 2 3 4 5 6 7 8 9 15412
25 10
```

Рис. 4 -Пример 1.

```
Primer_1.py Primer_2.py x
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import sys
5
6
7  if __name__ == '__main__':
8      a = list(map(int, input().split()))
9      if not a:
10         print('Заданный список пуст', file=sys.stderr)
11         exit(1)
12
13     a_min = a_max = a[0]
14     i_min = i_max = 0
15     for i, item in enumerate(a):
16         if item < a_min:
17             i_min, a_min = i, item
18
19         if item > a_max:
20             i_max, a_max = i, item
21
22     if i_min > i_max:
23         i_min, i_max = i_max, i_min
24
25     count = 0
26     for item in a[i_min+1:i_max]:
27         if item > 0:
28             count += 1
29
30     print(count)

Run Primer_2 x
C:\Users\Тимур\laba_4\.venv\Scripts\python.exe C:\Users\Тимур\laba_4\Primer_2.py
2 1 123 4 587 9999 12 3
3
```

Рис. 5 - Пример 2.

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import sys
5
6  if __name__ == '__main__':
7      A = tuple(map(int, input().split()))
8      if len(A) != 10:
9          print("Неверный размер списка", file=sys.stderr)
10         exit(1)
11
12         s = sum(a for a in A if abs(a) < 5)
13
14     print(s)
```

Run Primer_3

C:\Users\Тимур\laba_4\.venv\Scripts\python.exe C:\Users\Тимур\laba_4\Primer_3.py

1 2 3 4 5 6 7 8 9 10

10

Рис. 6 - Пример 3.

16. Ввести список A из 10 элементов, найти сумму элементов кратных 2, их количество и вывести результаты на экран.

Рис. 7 – индивидуальное задание 1 (В-16).

The image shows a Python IDE with a dark theme. At the top, there are three tabs: 'Primer_1.py', 'Primer_2.py', and 'individZadanie_1.py'. The 'individZadanie_1.py' tab is active, showing a Python script. The script starts with a shebang and a UTF-8 encoding declaration. It imports the 'sys' module. A main block is defined by 'if __name__ == '__main__':'. Inside, it prompts the user to enter 10 numbers, reads the input, and checks if the length is 10. If not, it prints an error to stderr and exits. If the length is 10, it initializes 'sum_even' and 'count_even' to 0. It then iterates over the list 'a' using a 'for' loop. For each item, it checks if it is a multiple of 2. If yes, it adds the item to 'sum_even' and increments 'count_even'. Finally, it prints the sum and count of even numbers.

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3
4 import sys
5
6 if __name__ == '__main__':
7     print("Enter 10 numbers separated by spaces: ")
8     a = list(map(int, input().split()))
9     if len(a) != 10:
10         print(f"Error: you need to enter exactly 10 numbers!", file=sys.stderr)
11         exit(1)
12
13     sum_even = 0
14     count_even = 0
15     for item in a:
16         if item % 2 == 0:
17             sum_even += item
18             count_even += 1
19
20     print(f"Sum of elements that are multiples of 2: {sum_even}")
21     print(f"Number of elements in multiples of 2: {count_even}")
```

Below the code editor, there is a 'Run' button and a terminal window. The terminal shows the command executed: 'C:\Users\Тимур\laba_4\.venv\Scripts\python.exe C:\Users\Тимур\laba_4\individZadanie_1.py'. The output of the program is displayed in the terminal: 'Enter 10 numbers separated by spaces:', followed by the input '1 2 3 4 5 6 7 8 9 10', then 'Sum of elements that are multiples of 2: 30', and finally 'Number of elements in multiples of 2: 5'.

Рис. 8- результат выполнения индивидуального задания 1. (циклы)

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3
4 import sys
5
6 if __name__ == '__main__':
7     print("Enter 10 numbers separated by spaces: ")
8     a = list(map(int, input().split()))
9     if len(a) != 10:
10         print(f"Error: you need to enter exactly 10 numbers!", file=sys.stderr)
11         exit(1)
12
13     e_el = [item for item in a if item % 2 == 0]
14
15     sum_even = sum(e_el)
16     count_even = len(e_el)
17
18     print(f"Elements that are multiples of 2: {e_el}")
19     print(f"The sum of the elements in multiples of 2: {sum_even}")
20     print(f"Number of elements in multiples of 2: {count_even}")
```

Run individZadanie_1(2) x

C:\Users\Тимур\laba_4\.venv\Scripts\python.exe "C:\Users\Тимур\laba_4\individZadanie_1(2).py"

Enter 10 numbers separated by spaces:

1 2 3 4 5 6 7 8 9 10

Elements that are multiples of 2: [2, 4, 6, 8, 10]

The sum of the elements in multiples of 2: 30

Number of elements in multiples of 2: 5

Рис. 9- результат выполнения индивидуального задания 1. (List Comprehensions)

16. В списке, состоящем из вещественных элементов, вычислить:

1. количество положительных элементов списка;
2. сумму элементов списка, расположенных после последнего элемента, равного нулю.

Рис. 10 – индивидуальное задание 2 (В-16).


```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys

if __name__ == '__main__':
    a = list(map(float, input().split()))

    if not a:
        print("The specified list is empty", file=sys.stderr)
        exit(1)

    count_pos = 0
    for num in a:
        if num > 0:
            count_pos += 1

    last_zero_index = -1
    for i in range(len(a) - 1, -1, -1):
        if a[i] == 0:
            last_zero_index = i
            break

    sum_after_zero = 0
    if last_zero_index != -1 and last_zero_index < len(a) - 1:
        # Берем срез списка после последнего нуля
        after_zero = a[last_zero_index + 1:]
        for num in after_zero:
            sum_after_zero += num
    elif last_zero_index == -1:
        for num in a:
            sum_after_zero += num

    print(f"The number of positive elements: {count_pos}")
    print(f"The sum of the elements after the last zero: {sum_after_zero}")
```

Run individZadanie_2

```
C:\Users\Тимур\laba_4\.venv\Scripts\python.exe C:\U
1 2 3 4 5 -1 -2 0 1 2
The number of positive elements: 7
The sum of the elements after the last zero: 3.0
Process finished with exit code 0
```

Рис. 11- результат выполнения индивидуального задания 2.

командой в двух терминалах.

16. Известны данные о мощности двигателя (в лошадиных силах – л. с.) и стоимости 30 марок легковых автомобилей. Напечатать стоимость каждого из автомобилей, у которых мощность двигателя не превышает 80 л. с.

Рис. 12 - индивидуальное задание 3 (В-16).

```
Primer_1.py Primer_2.py Primer_3.py individZadanie_1.py individZadanie_2.py individZadanie_3.py x
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3
4 import sys
5
6 if __name__ == '__main__':
7     power = tuple(map(int, input("Powers (30 numbers): ").split()))
8     price = tuple(map(int, input("Cost (30 numbers): ").split()))
9
10    if len(power) != 30 or len(price) != 30:
11        print("You need exactly 30 values.", file=sys.stderr)
12        exit(1)
13
14    count = 0
15    print("Car prices ≤ 80 horsepower:")
16    for pwr, prc in zip(power, price):
17        if pwr <= 80:
18            print(f"{prc} k.$")
19            count += 1
20
21    print(f"Total found: {count} cars")
```

Run individZadanie_3 x

```
C:\Users\Тимур\laba_4\.venv\Scripts\python.exe C:\Users\Тимур\laba_4\individZadanie_3.py
Powers (30 numbers): 1 2 3 4 5 6 7 8 9 100 111 12 13 14 155 166 177 188 199 200 211 222 233 244 255 266 277 288 299 300
Cost (30 numbers): 1 2 3 4 5 6 7 8 9 100 111 122 13 14 15 166 177 188 199 200 211 222 233 244 255 266 277 288 299 300
Car prices ≤ 80 horsepower:
1 k.$
2 k.$
3 k.$
4 k.$
5 k.$
6 k.$
7 k.$
8 k.$
9 k.$
122 k.$
13 k.$
14 k.$
17 k.$
188 k.$
199 k.$
24 k.$
```

Рис. 13- результат выполнения индивидуального задания 3.

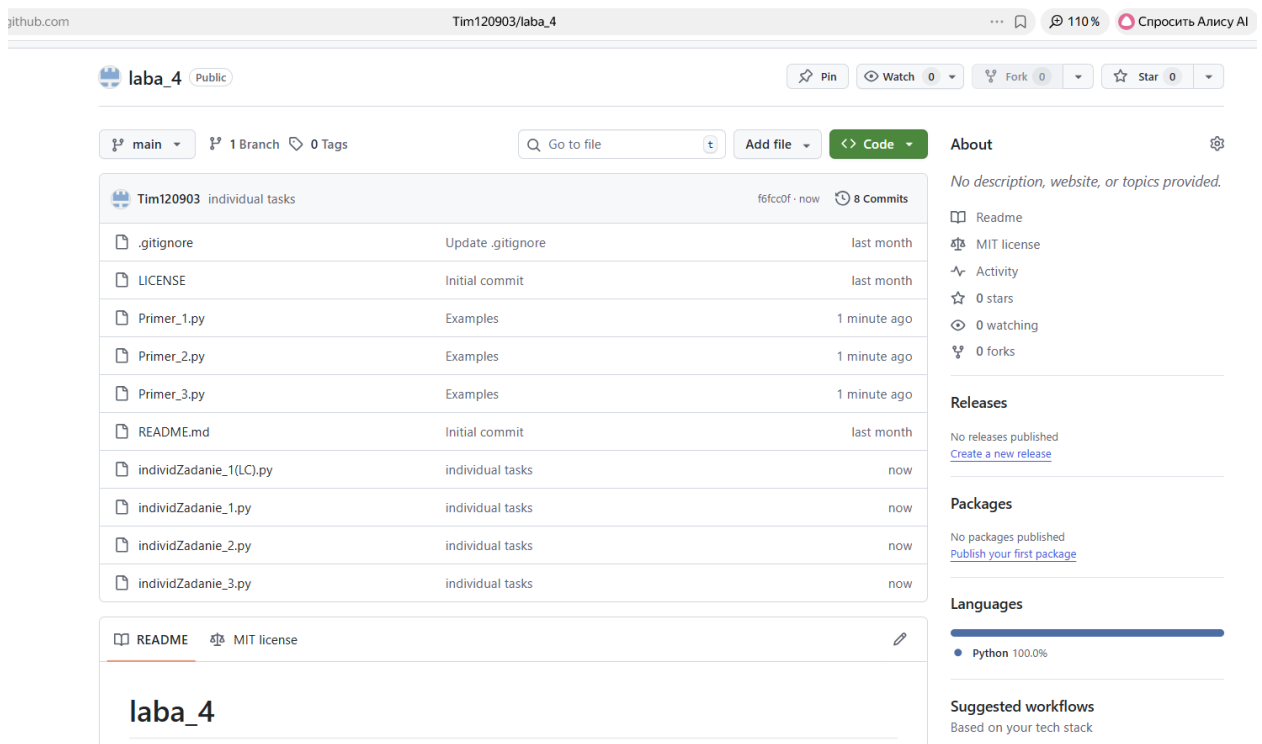


Рис. 14 - добавленные файлы с коммитами на Github.

Ответы на контрольные вопросы:

1) Список (list) – это структура данных для хранения объектов различных типов. Размер списков не статичен, его можно изменять. Список по своей природе является изменяемым типом данных. Переменная, определяемая как список, содержит ссылку на структуру в памяти, которая в свою очередь хранит ссылки на какие-либо другие объекты или структуры.

2) Для создания списка нужно элементы заключить в квадратные скобки: `my_list = [1, 2, 3, 4, 5]`

3) При создании списка в памяти резервируется область, которую можно условно назвать некоторым “контейнером”, в котором хранятся ссылки на другие элементы данных в памяти. В отличие от таких типов данных как число или строка, содержимое “контейнера” списка можно менять.

4) Перебрать все элементы списка можно с помощью следующего цикла: `my_list = ['Один', 'Два', 'Три', 'Четыре', 'Пять']`
`for elem in my_list:`
`print(elem)`

5) Арифметические операции со списками: `+` (объединение списков), `*` (повтор списка).

6) Для того, чтобы проверить, есть ли заданный элемент в списке Python необходимо использовать оператор `in`:

7) Метод `count` можно использовать для определения числа вхождений заданного элемента в списке.

8) Метод `insert` можно использовать для вставки элемента в список. Метод `append` можно использовать для добавления элемента в список.

9) Для сортировки списка нужно использовать метод `sort`.

10) Удалить элемент можно, написав его индекс в методе `pop`.

11) List Comprehensions (абстракция списков или списковое включение) является частью синтаксиса языка, которая предоставляет простой способ построения списков. В языке Python есть 2 очень мощные функции для работы с коллекциями: `map` и `filter`. Они позволяют использовать функциональный стиль программирования, не прибегая к помощи циклов, для работы с такими

типами как list, tuple, set, dict и т.п. Списковое включение позволяет обойтись без этих функций.

12) Срезы задаются тройкой чисел, разделенных запятой: start, stop, step. Start – позиция, с которой нужно начать выборку, stop – конечная позиция, step – шаг. При этом необходимо помнить, что выборка не включает элемент определяемый stop.

13) Для работы со списками существуют следующие функции агрегации: – len(L) – получить число элементов в списке – min(L) – получить минимальный элемент списка – max(L) – получить максимальный элемент списка – sum(L) – получить сумму элементов списка, если список содержит только числовые значения.

14) Для создания копии списка необходимо использовать либо метод copy, либо оператор среза.

15) Функция sorted() в Python создаёт новый отсортированный список, не изменяя исходный. Это полезно, когда необходимо сохранить оригинальный порядок элементов. Отличие от метода sort() списков заключается в том, что метод sort() изменяет исходный список на месте и возвращает None. Этот подход эффективен по памяти, поскольку не создаёт новый список. Ещё одно отличие в том, что метод list.sort() определён только для списков, в то время как функция sorted() работает со всеми итерируемыми объектами (списки, кортежи, строки, множества, словари и т. д.).

16) Кортеж (tuple) – это неизменяемая структура данных, которая по своему подобию очень похожа на список.

17) Существует несколько причин, по которым стоит использовать кортежи вместо списков. Одна из них - это обезопасить данные от случайного изменения. Если мы получили откуда-то массив данных, и у нас есть желание поработать с ним, но при этом непосредственно менять данные мы не собираемся, тогда, это как раз тот случай, когда кортежи придутся как нельзя кстати. Используя их в данной задаче, мы дополнительно получаем сразу несколько бонусов - во-первых, это экономия места. Дело в том, что кортежи

в памяти занимают меньший объем по сравнению со списками. Во-вторых - прирост производительности, который связан с тем, что кортежи работают быстрее, чем списки (т. е. на операции перебора элементов и т. п. будет тратиться меньше времени). Важно также отметить, что кортежи можно использовать в качестве ключа у словаря.

18) Кортеж создается также как список, только вместо квадратных скобок используют круглые. При желании можно воспользоваться функцией `tuple()`. Для создания пустого кортежа, не нужно ничего писать в скобках.

19) Доступ к элементам кортежа осуществляется также как и к элементам списка – через указание индекса. Но изменять элементы кортежа нельзя.

20) Обращение по индексу, это не самый удобный способ работы с кортежами. Дело в том, что кортежи часто содержат значения разных типов, и помнить, по какому индексу что лежит – очень непросто. Для этого существует деструктуризация.

21) Благодаря тому, что кортежи легко собирать и разбирать, в Python удобно делать такие вещи, как множественное присваивание: `(a, b, c) = (1, 2, 3)`

22) С помощью операции взятия среза можно получить другой кортеж: `T2 = T1[i:j]`.

23) Для кортежей можно выполнять операцию конкатенации, которая обозначается символом `+`. В простейшем случае общая форма операции следующая: `T3 = T1 + T2`. Кортеж может быть образован путем операции повторения, обозначаемой символом `*`. Общая форма: `T2 = T1 * n` (где `n` – количество повторений кортежа `T1`).

24) Элементы кортежа можно последовательно просмотреть с помощью операторов цикла `while` или `for`, например: `A = ("abc", "abcd", "bcd", "cde")` `for item in A: print(item)`

25) Для проверки вхождения элемента в кортеж используется операция `in`: `A = ("abc", "abcd", "bcd", "cde")` `item = str(input("s = "))` `if (item in A):` `print(item, " in ", A, "= True")`

26) Методы работы с кортежем: – `index()` – поиск позиции элемента в кортеже `count()` – количество вхождения элемента в кортеж

27) Да, использование функций агрегации (таких как `len()`, `sum()`, `max()`, `min()` и др.) с кортежами в Python полностью допустимо и корректно. Кортежи поддерживают те же операции, что и другие итерируемые объекты (например, списки), так как они являются последовательностями (sequence type).

28) Создание кортежа с помощью спискового включения: `a = tuple(map(int, input().split()))`.

Вывод: в ходе работы были приобретены навыки по работе со списками и кортежами при написании программ с помощью языка программирования Python версии 3.x.